Transformers
(Part II)

Two Ways of using Transformers

- A Transformer network can be used as *feature transform or as sequence modeling*
- Transformers for **sequence modeling**

$$h_t = f\ (x_t, h_{0:t-1}\ ) \rightarrow h_{1:t} = \Phi(x_{1:t}, h_0)$$

- Transformers for **feature transform**

$$z_{1:t} = f\ (x_{1:t})$$

Two Ways of using Transformers

- A Transformer network can be used as *feature transform or as sequence modeling*
- Transformers for **sequence modeling**

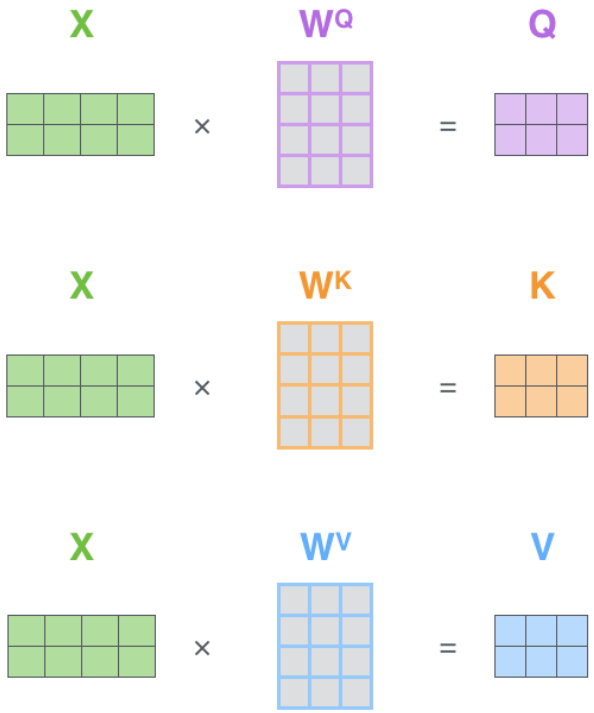
$$h_t = f\ (x_t, h_{0:t-1}\ ) \rightarrow h_{1:t} = \Phi(x_{1:t}, h_0)$$

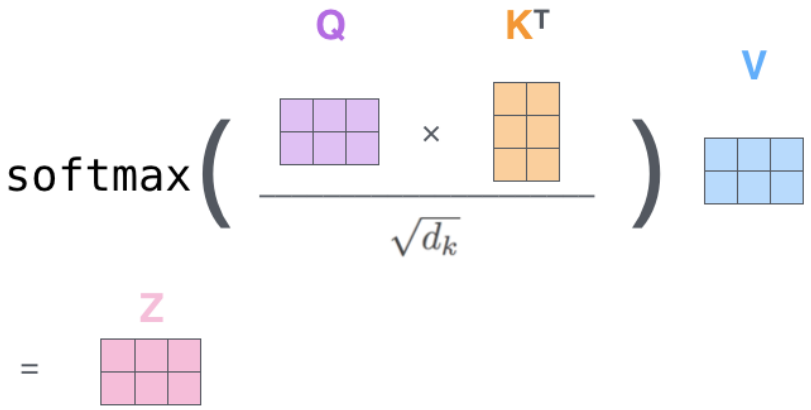
- Transformers for **feature transform**

$$z_{1:t} = f\ (x_{1:t})$$

Self-Attention for Feature Transform

- Input: a 1D feature map $X \in R^{T \times d}$
- Output: transformed 1D feature map $z \in R^{T \times d_k}$





Slide Credit: Jay Alammar. [The Illustrated Transformer]

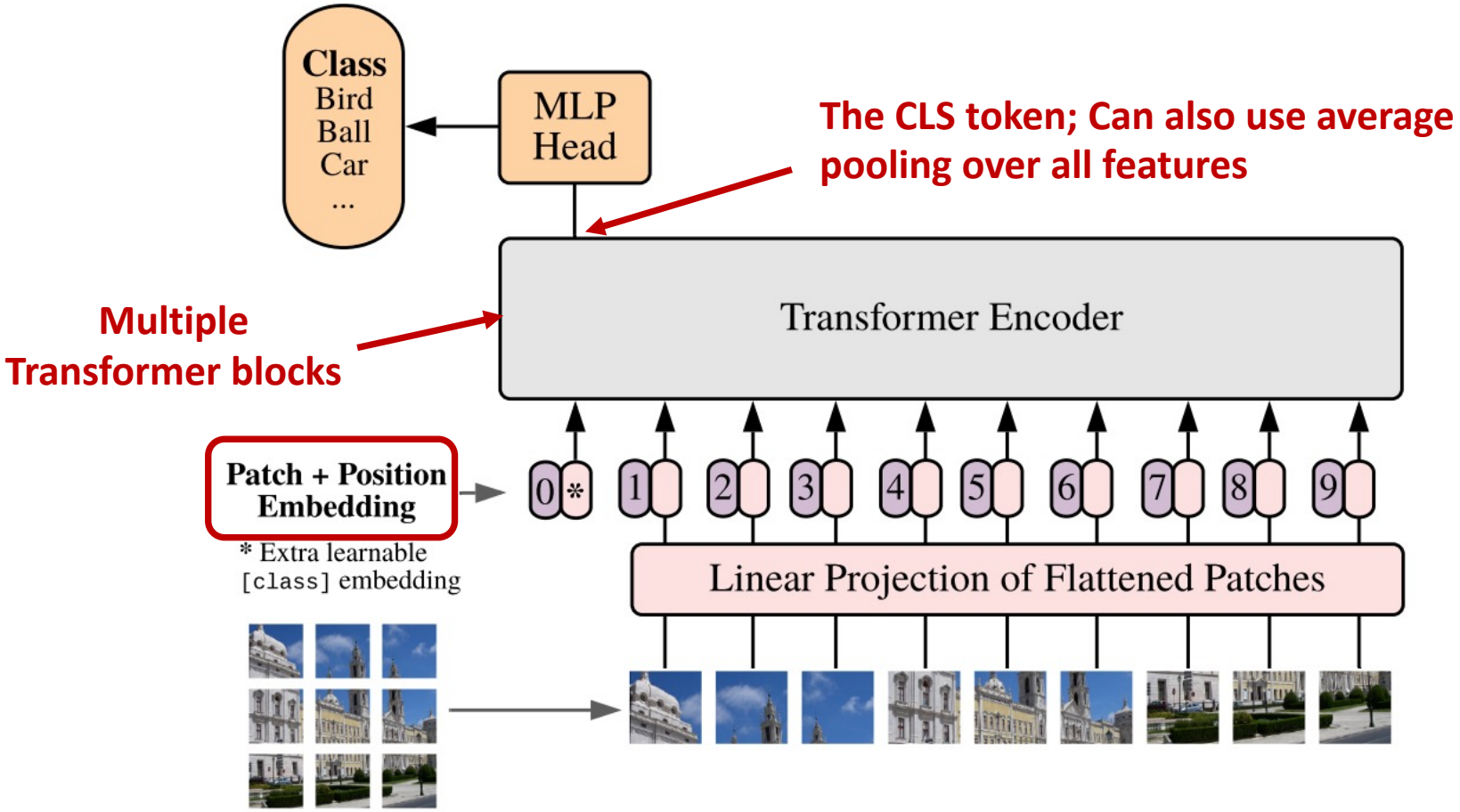
Vision Transformer

- Similar to a convolutional network, we can build an image classification model using Transformers
- **Key idea:** treating an input image as a sequence of *patches*, akin to a string of *words*

Vision Transformer

Source: https://research.google/blog/transformers-for-image-recognition-at-scale/

Vision Transformer



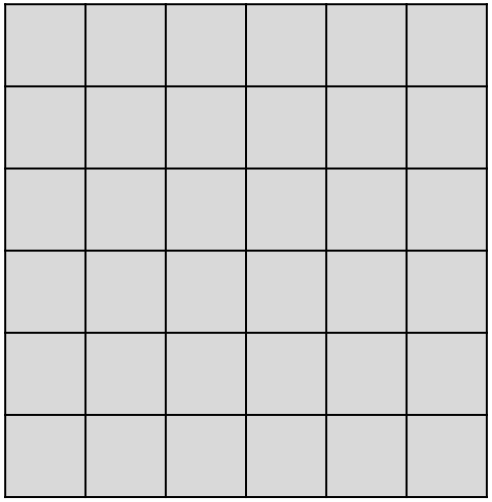
Dosovitskiy, A., Beyer, L., Kolesnikov, A., Weissenborn, D., Zhai, X., Unterthiner, T., ... & Houlsby, N. (2021). An image is worth 16x16 words: Transformers for image recognition at scale. *ICLR*

Vision Transformer: Summary

- **Key idea:** treating an input image as a sequence of *patches*, akin to a string of *words*
  - Patches are embedded using linear projection (**vision tokens**)
  - Positional encoding is added to patch embedding
  - Use an extra learnable CLS token for classification
- **Complexity**: $O(T^2)$ in compute / memory with T as the number of patches due to **self-attention**

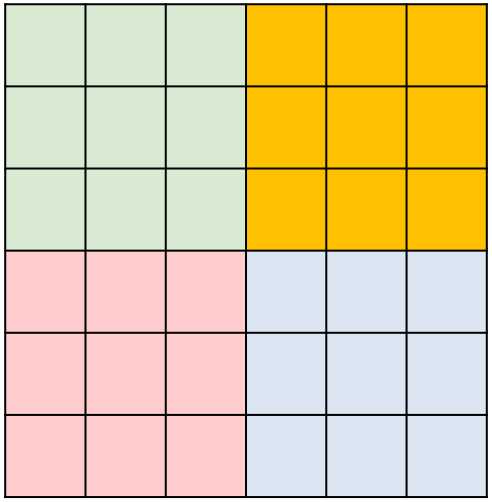
Global vs. Local Self-Attention

- **Global** self-attention
  - Pairwise attention scores: each position must attend to all positions
  - Attention scores are normalized using Softmax
  - $O(T^2)$ in compute / memory



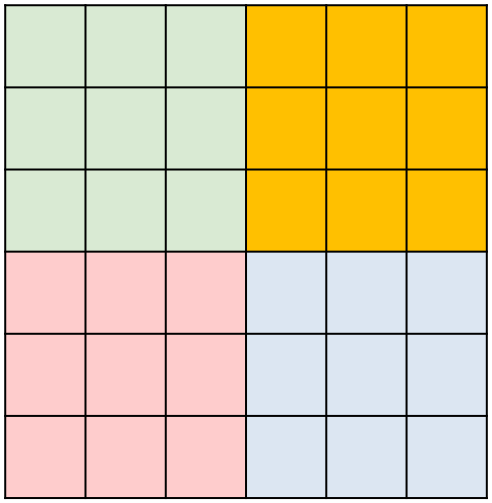
Global vs. Local Self-Attention

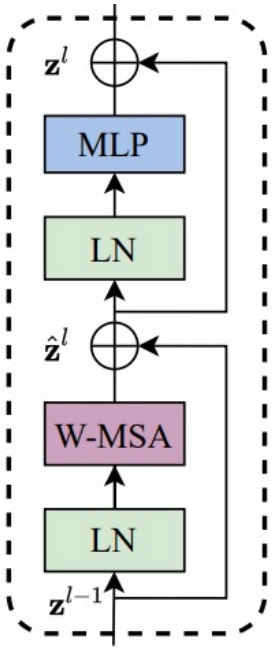
- **Local** self-attention
  - Attention computed in non-overlapping local windows
  - Significantly reduced computational cost (why?)



Local Self-Attention

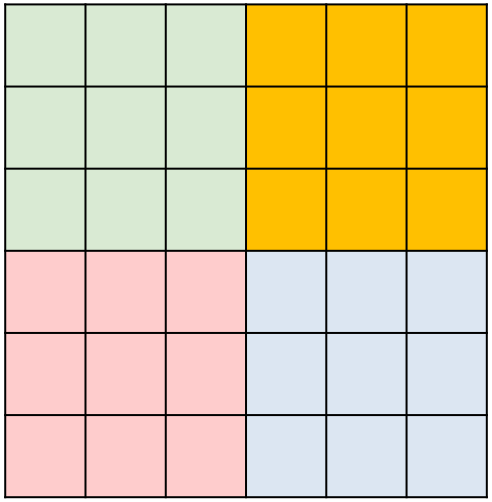
- Local self-attention
  - Attention computed in non-overlapping local windows
  - Significantly reduced computational cost
  - Transformer blocks with local self-attention (W-MSA)

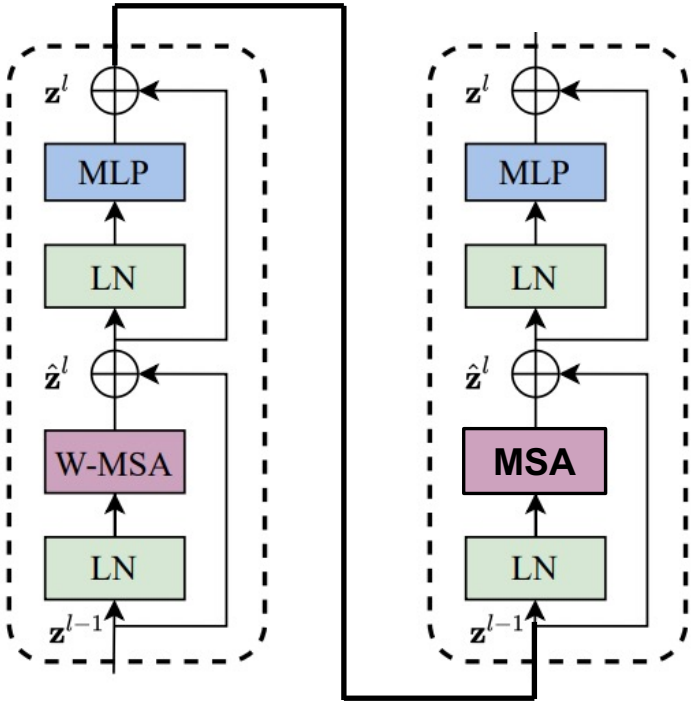




Beyond Local Self-Attention

- Transformer blocks with local self-attention
  - Limited receptive field (patches within the same window)
  - Design: Interleaving global / local attention

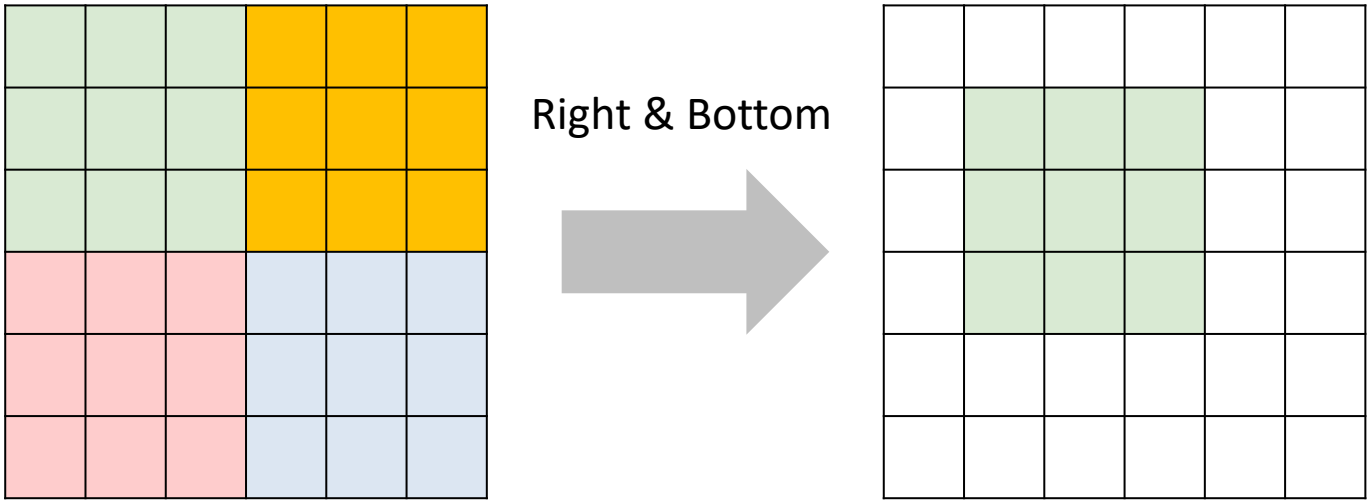




Li, Y., Mao, H., Girshick, R., & He, K. (2022). Exploring plain vision transformer backbones for object detection. In *ECCV*

“Shifted” Local Self-Attention

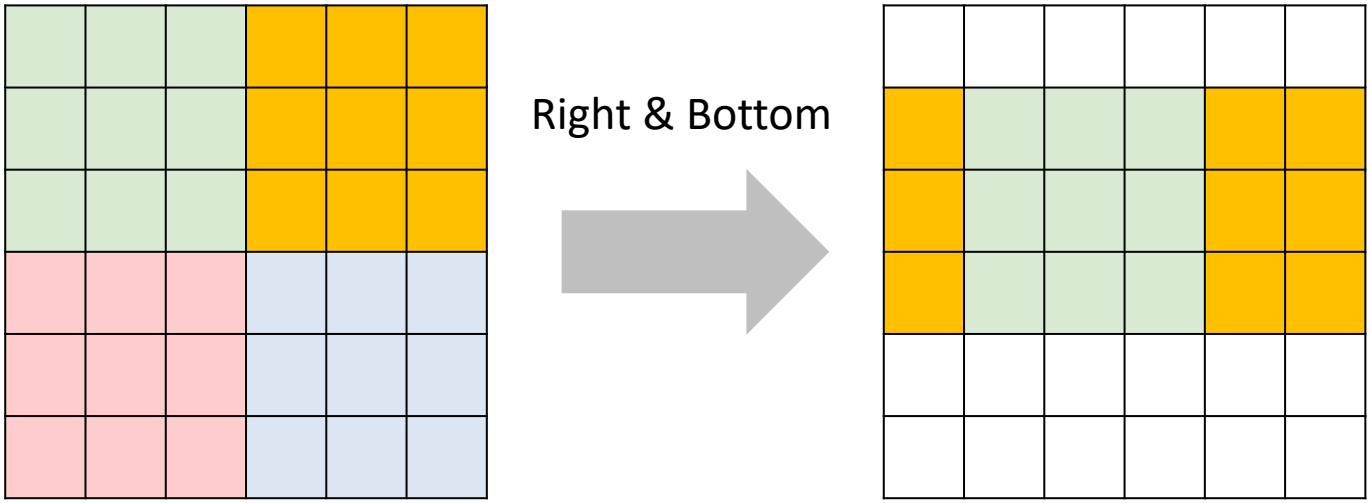
- Local self-attention
  - Attention computed in non-overlapping local windows
- Shift the feature map
  - = circular shift of the local window



Liu, Z., Lin, Y., Cao, Y., Hu, H., Wei, Y., Zhang, Z., ... & Guo, B. (2021). Swin transformer: Hierarchical vision transformer using shifted windows. In *CVPR*

“Shifted” Local Self-Attention

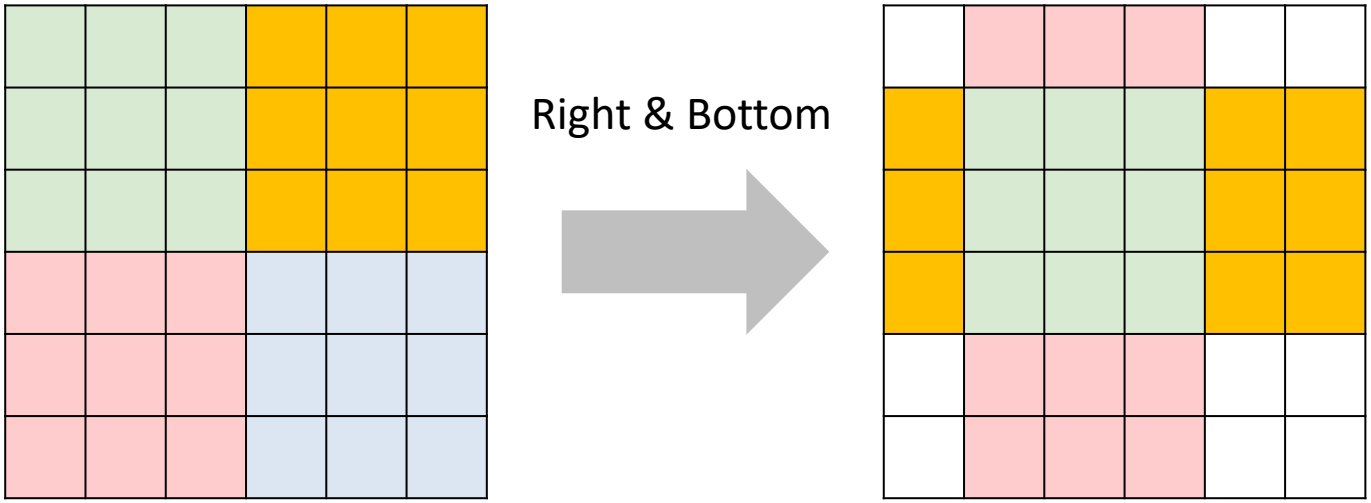
- Local self-attention
  - Attention computed in non-overlapping local windows
- Shift the feature map
  - = circular shift of the local window



Liu, Z., Lin, Y., Cao, Y., Hu, H., Wei, Y., Zhang, Z., ... & Guo, B. (2021). Swin transformer: Hierarchical vision transformer using shifted windows. In *CVPR*

“Shifted” Local Self-Attention

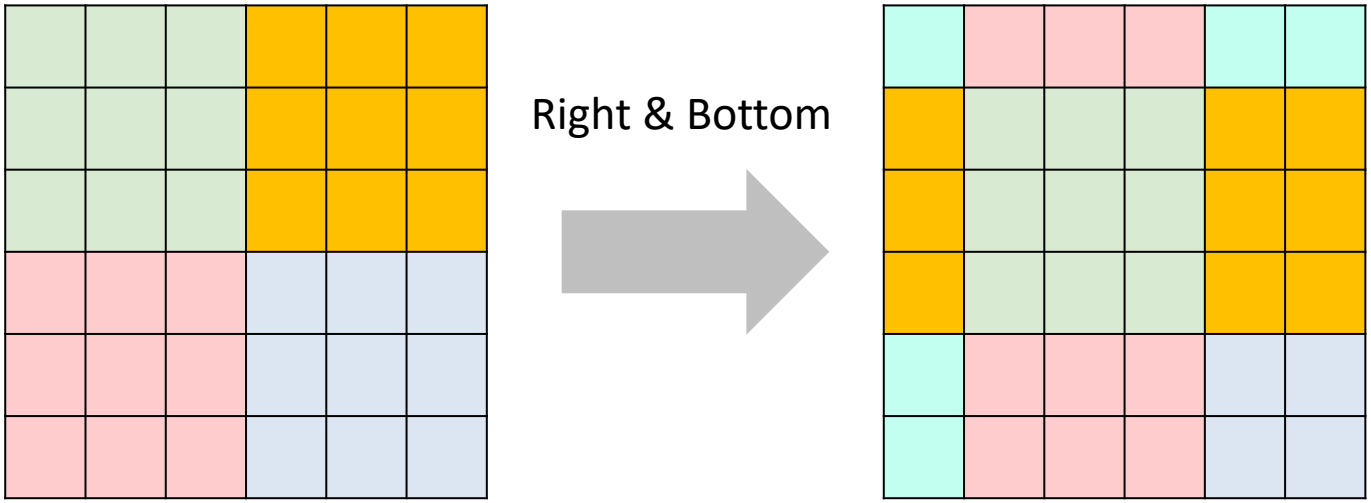
- Local self-attention
  - Attention computed in non-overlapping local windows
- Shift the feature map
  - = circular shift of the local window



Liu, Z., Lin, Y., Cao, Y., Hu, H., Wei, Y., Zhang, Z., ... & Guo, B. (2021). Swin transformer: Hierarchical vision transformer using shifted windows. In *CVPR*

“Shifted” Local Self-Attention

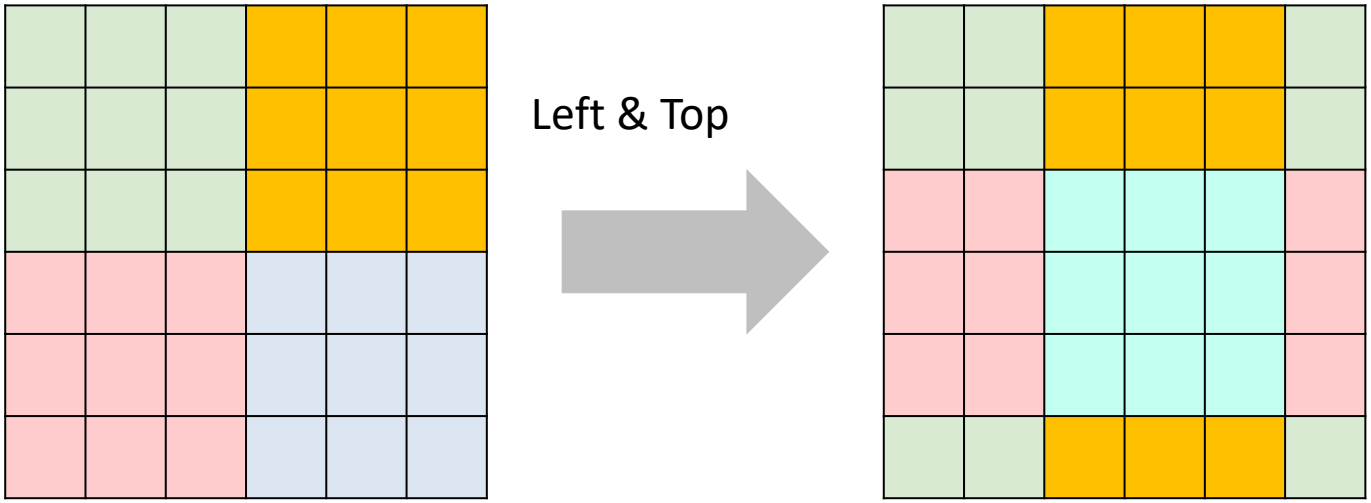
- Local self-attention
  - Attention computed in non-overlapping local windows
- Shift the feature map
  - = circular shift of the local window



Liu, Z., Lin, Y., Cao, Y., Hu, H., Wei, Y., Zhang, Z., ... & Guo, B. (2021). Swin transformer: Hierarchical vision transformer using shifted windows. In *CVPR*

“Shifted” Local Self-Attention

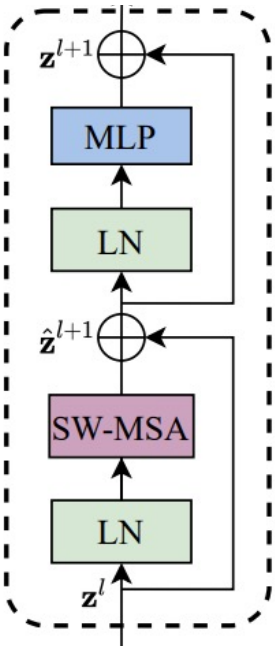
- Local self-attention
  - Attention computed in non-overlapping local windows
- Shift the feature map
  - = circular shift of the local window



Liu, Z., Lin, Y., Cao, Y., Hu, H., Wei, Y., Zhang, Z., ... & Guo, B. (2021). Swin transformer: Hierarchical vision transformer using shifted windows. In *CVPR*

“Shifted” Local Self-Attention

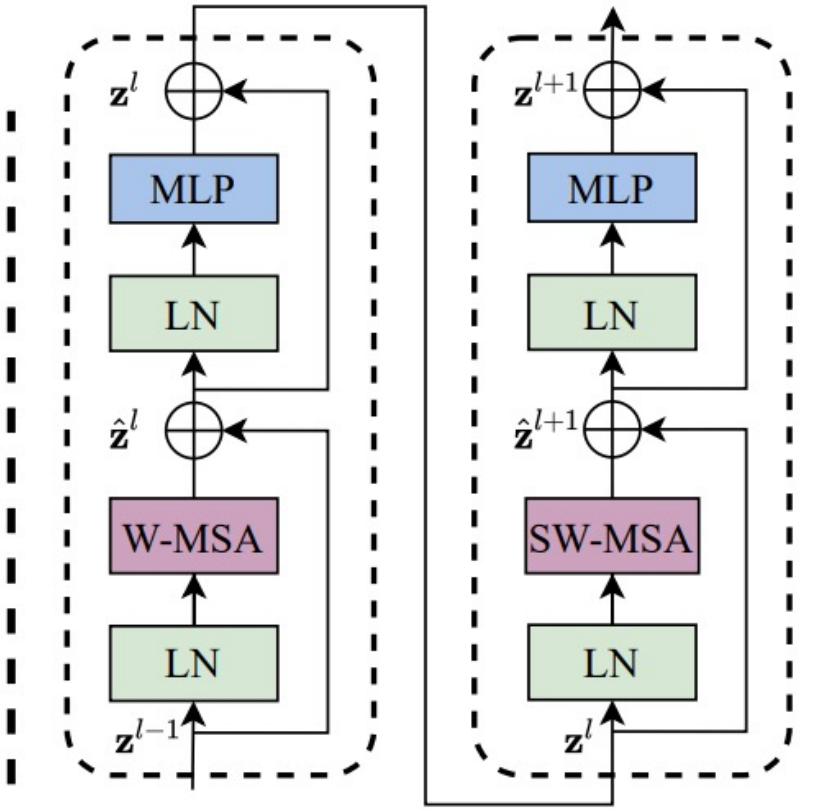
- Local self-attention
  - Attention computed in non-overlapping local windows
- Shift the feature map
  - = circular shift of the local window
  - Transformer blocks with shifted local self attention
  - Can be paired with local self-attention to allow features to interact beyond local window



Liu, Z., Lin, Y., Cao, Y., Hu, H., Wei, Y., Zhang, Z., ... & Guo, B. (2021). Swin transformer: Hierarchical vision transformer using shifted windows. In *CVPR*

Swin Transformer Block

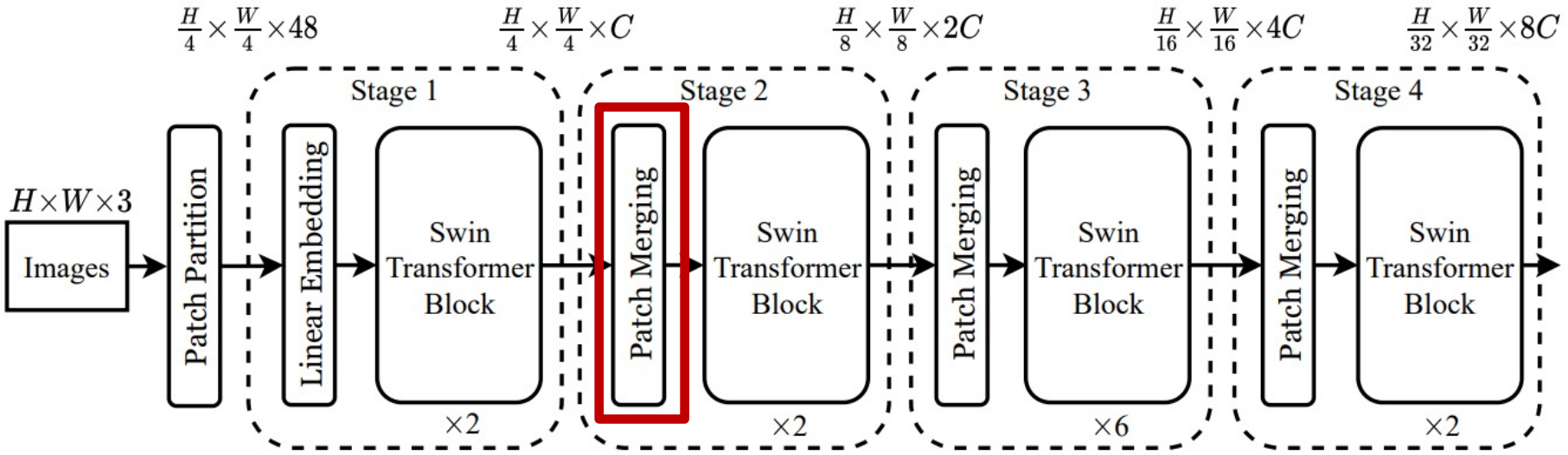
- A Transformer block with local self-attention
- A Transformer block with “shifted” local self-attention
- Learned relative position embedding added to the self-attention



Two Successive Swin Transformer Blocks

Liu, Z., Lin, Y., Cao, Y., Hu, H., Wei, Y., Zhang, Z., ... & Guo, B. (2021). Swin transformer: Hierarchical vision transformer using shifted windows. In *CVPR*

Swin Transformer



Downsampling using a strided convolution

Liu, Z., Lin, Y., Cao, Y., Hu, H., Wei, Y., Zhang, Z., ... & Guo, B. (2021). Swin transformer: Hierarchical vision transformer using shifted windows. In *CVPR*

Recap: Ways of using Transformers

- A Transformer network can be used as *feature transform or as sequence modeling*
- Transformers for **sequence modeling**

$$h_t = f\ (x_t, h_{0:t-1}\ ) \rightarrow h_{1:t} = \Phi(x_{1:t}, h_0)$$

- Transformers for **feature transform**

$$z_{1:t} = f\ (x_{1:t})$$

Example: ChatGPT

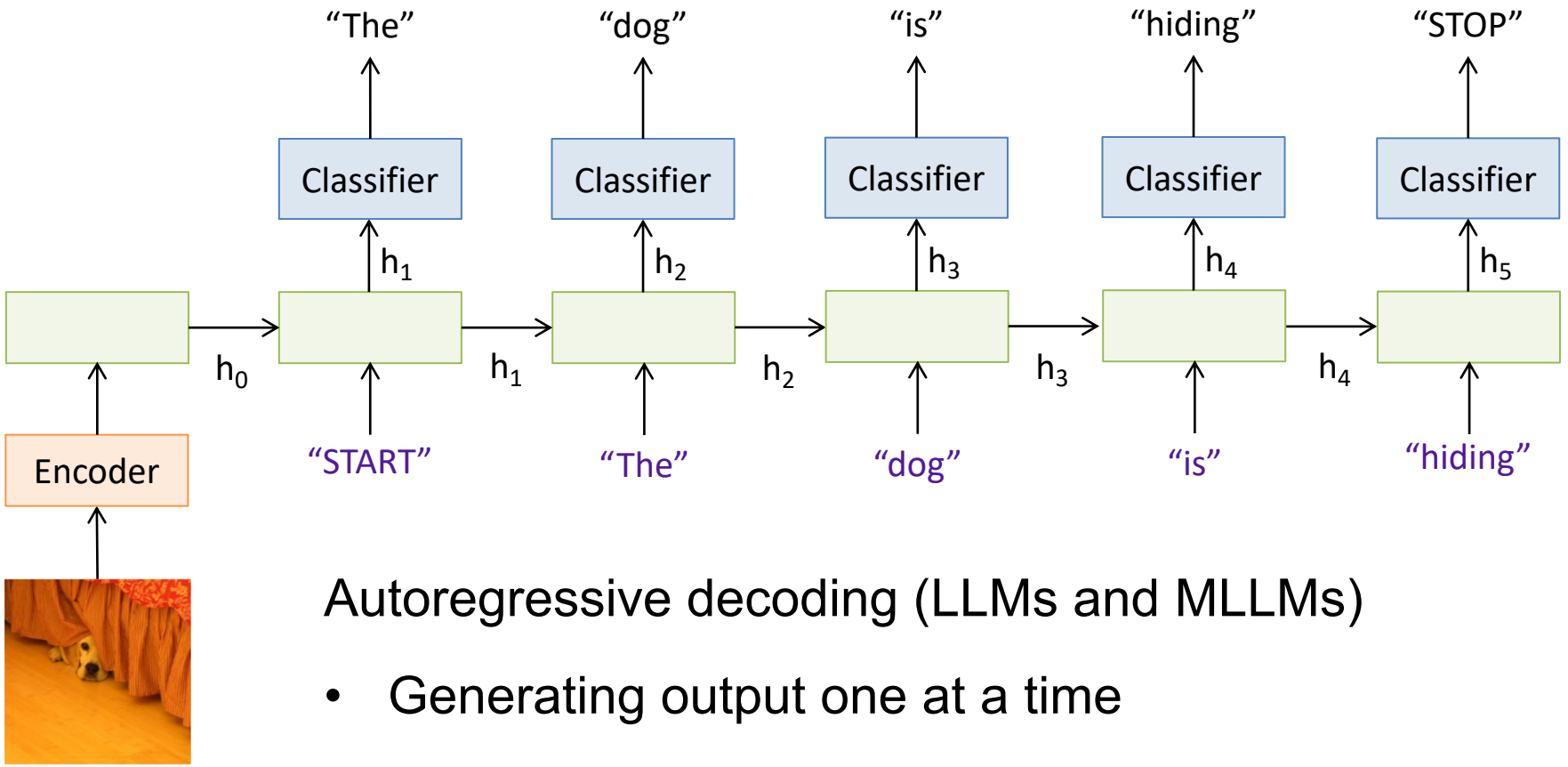
Y



What is inside of this image?

The image shows a group of people gathered around a campfire during nighttime. They appear to be engaged in a conversation or perhaps listening to a story, given their attentive postures. The glow of the fire illuminates their faces, creating a warm and inviting atmosphere. In the background, the lights of a city or town can be seen, suggesting that this outdoor scene is not far from an urban area. The presence of trees silhouetted against the night sky adds to the outdoor, wilderness vibe of the setting.

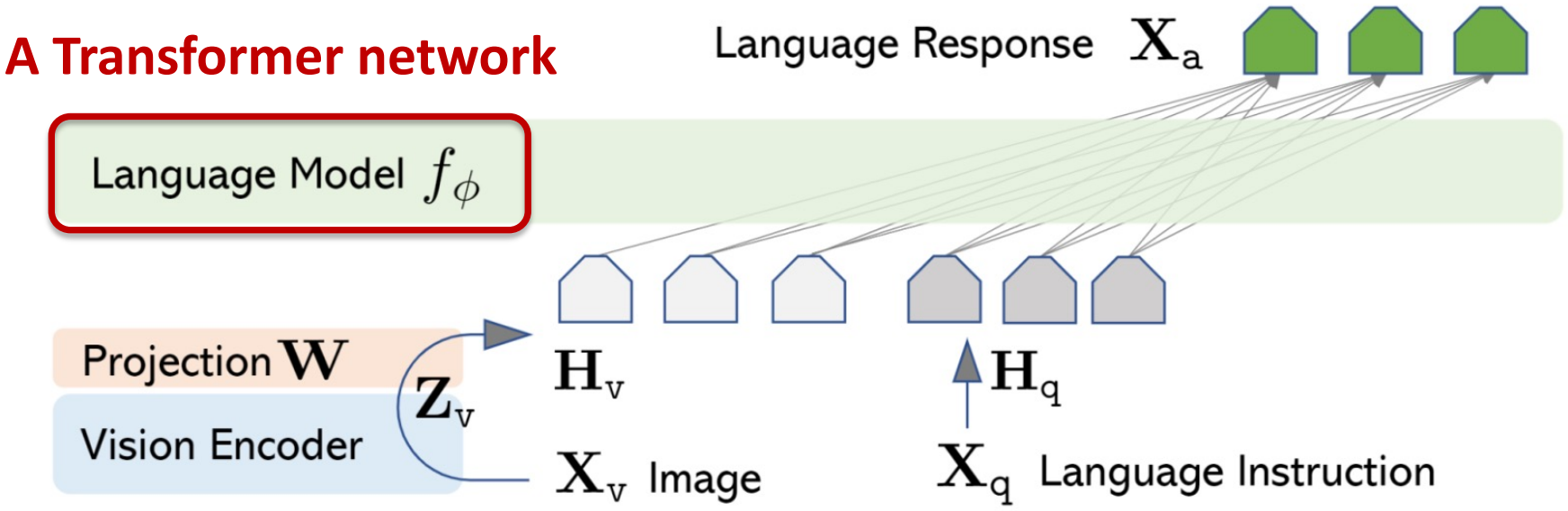
Recall: Image Caption Generation



Autoregressive decoding (LLMs and MLLMs)

- Generating output one at a time
- Feeding model output at $t-1$ as the input at t

A Modern Approach (LLaVA)



- Combine vision and text tokens
- General framework to support multiple vision tasks
- More in our lectures on generative models

H. Liu, C. Li, Q. Wu, Y. J. Lee, Visual Instruction Tuning, NeurIPS 2023

Self-Attention for Sequence Modeling

- **Prefill stage** = feature transform
  - Input: a 1D feature map $X \in R^{T \times d}$ (e.g., combined vision and text tokens)
  - Output: transformed 1D feature map $z \in R^{T \times d_k}$ (**KV Cache**)

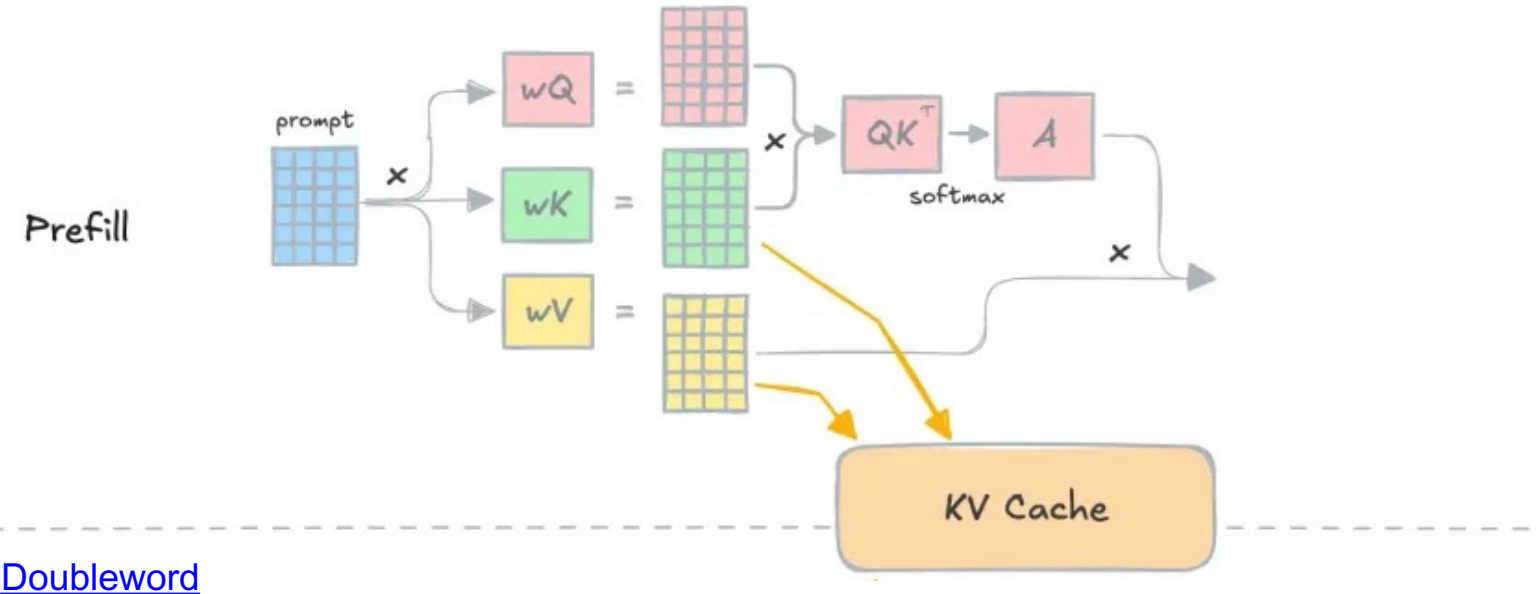


Figure Credit: Doubleword

Self-Attention for Sequence Modeling

- **(Autoregressive) decoding stage**
  - Input: a 1D feature vector $x \in R^{1\times d}$
  - Output: transformed 1D feature vector $z \in R^{1\times d_k}$ (which is often further decoded into a **text token**); *Also* **updated KV Cache**

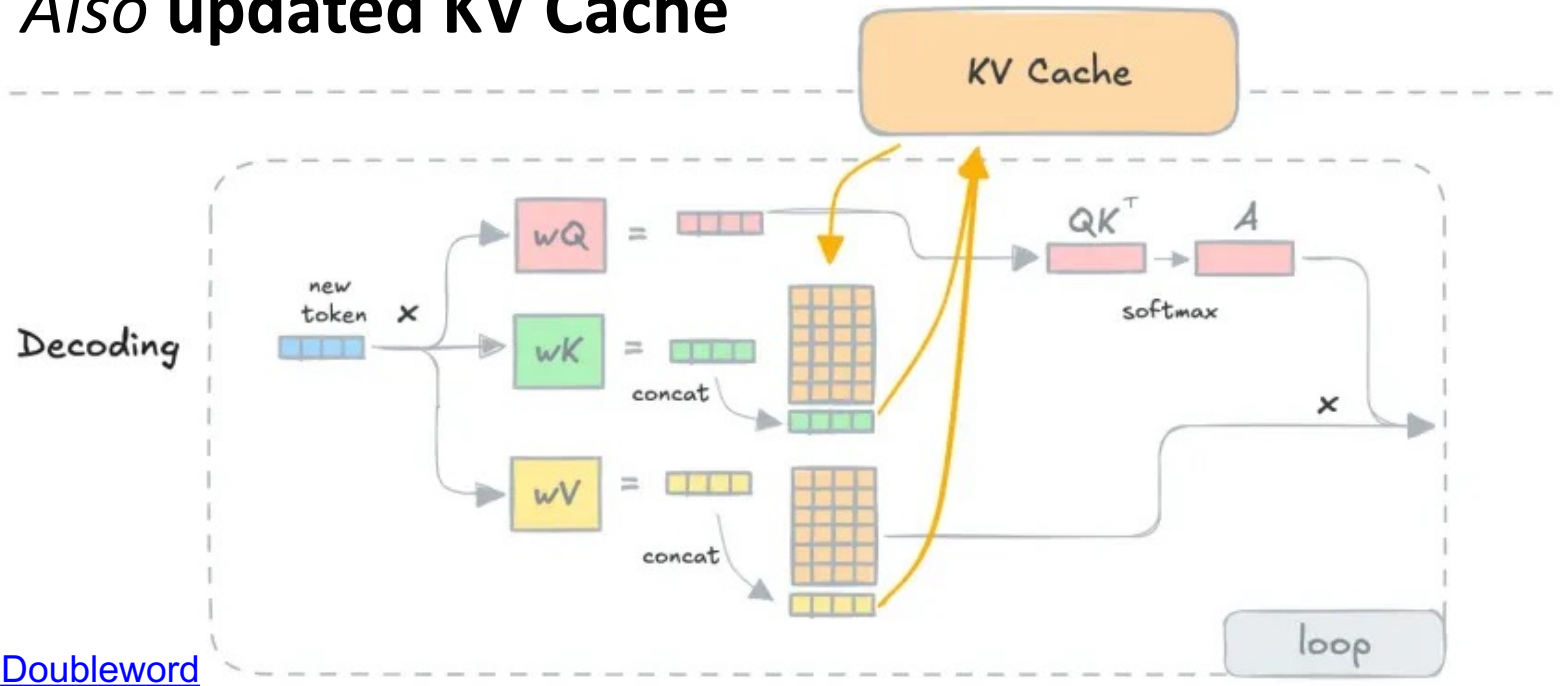


Figure Credit: Doubleword

Self-Attention for Sequence Modeling

Parallel processing; compute bound

Prefill

$O(T^2)$ in compute and memory (once)

Sequential processing; memory bound

Decoding

$O(T)$ in compute and memory (many times)

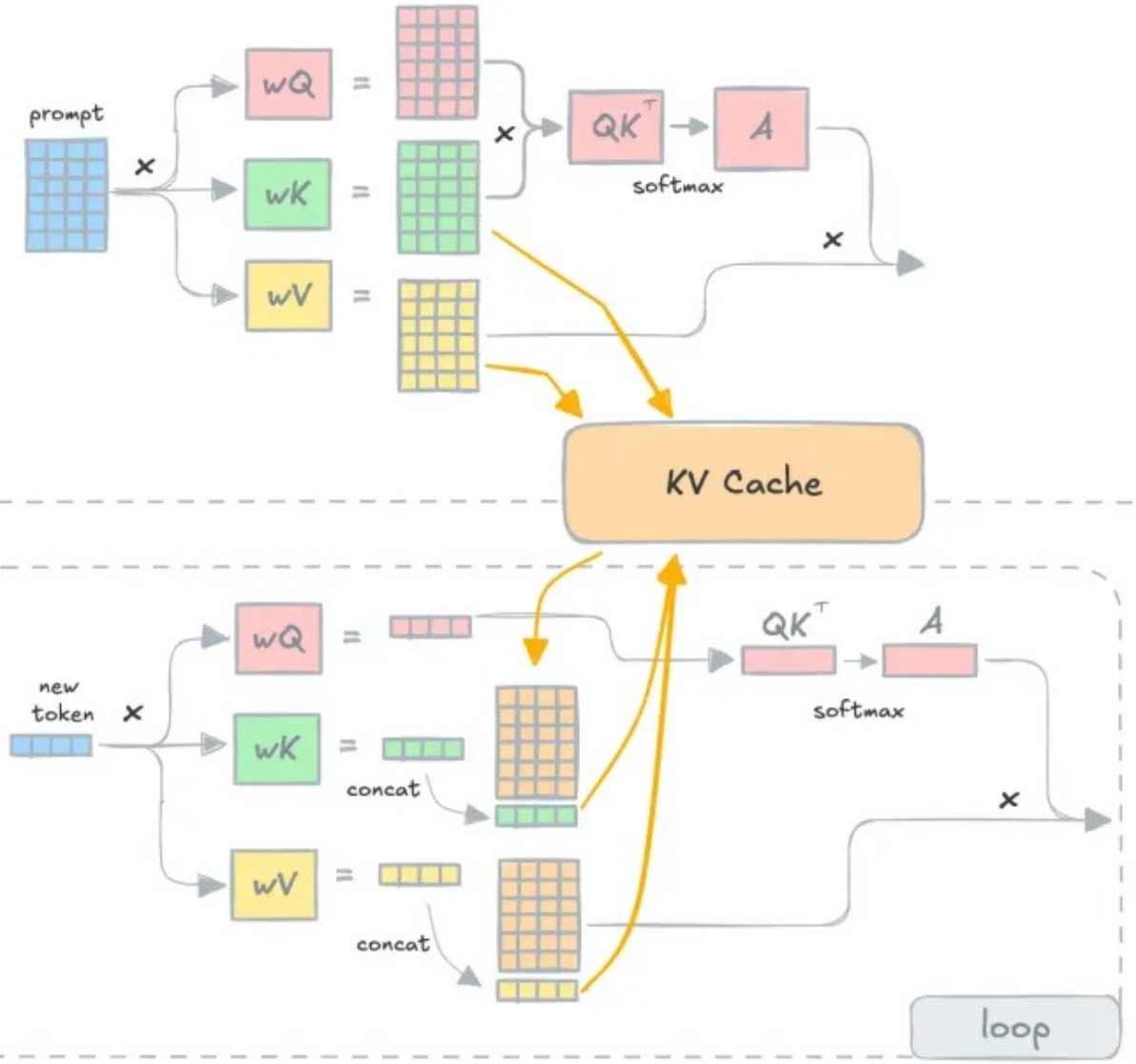


Figure Credit: Doubleword

Two Ways of using Transformers

- A Transformer network can be used as *feature transform or as sequence modeling*
- Transformers for **sequence modeling**

$$h_t = f\ (x_t, h_{0:t-1}\ ) \rightarrow h_{1:t} = \Phi(x_{1:t}, h_0)$$

- Transformers for **feature transform**

$$z_{1:t} = f\ (x_{1:t})$$

Notable Transformer Architecture Variants

- Better position encoding by considering relative positioning: RoPE

- Efficient “attention”
  - Linear attention: DeltaNet / Gated DeltaNet (also see structure stat space models: Mamba / Mamba2)
  - Group-query attention (GQA)

Transformers: Summary

- A Transformer block = self-attention + layer norm + MLP / FFN (w. residual connections)
- A Transformer network is built on Transformer blocks
- A Transformer network can be used for *feature transform or for sequence modeling*
- Capable of capturing long-range dependency
- Transformer variants are now behind many AI solutions